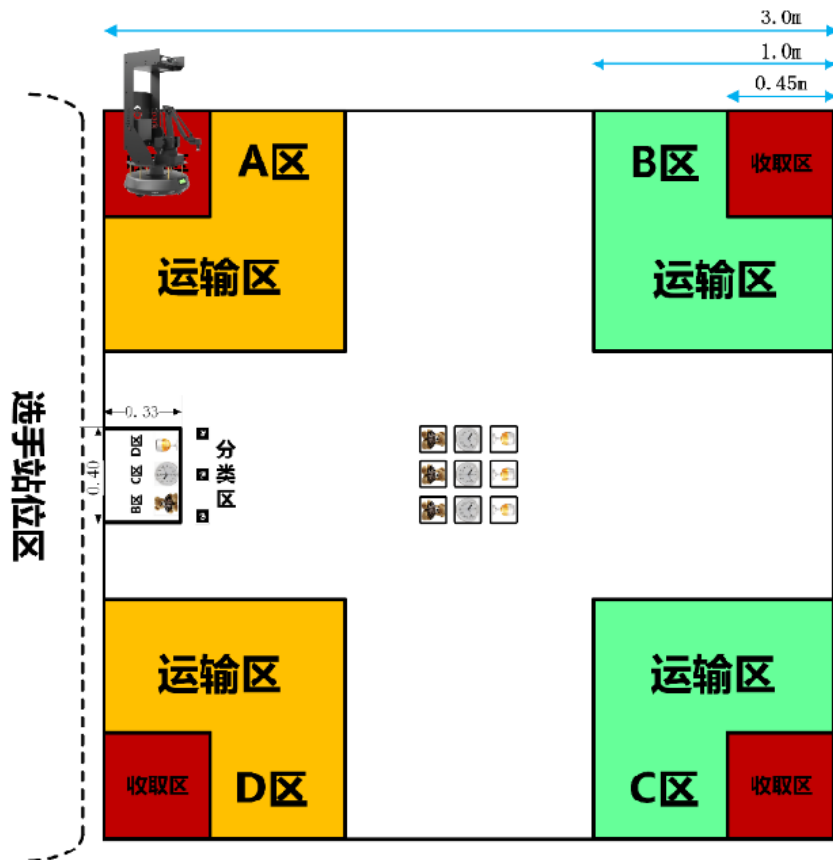


【夺宝奇兵自动赛-说明文档】

1. 比赛说明



🌟 地图说明:

得分区

竞赛场地的四个角落划分了B、C、D三个得分区，每个得分区为1m X 1m的正方形。

收取区

每个得分区内都划分了一个大小为0.45m X 0.45m的收取区，收取区为红色。

运输区

每个得分区除收取区以外的为运输区。其中A、D运输区为黄色，B、C运输区为绿色。收取区和运输区之间以黑色线条划分，黑色线条外线以内属于收取区，外线以外属于运输区。

分类区

在A区与C区中间，放置分类区，尺寸：40cm(长) x 33cm(宽) x 20cm(高)，分类区台子上摆放区域与得分方块的对应关系。分类区放置台附近，放置了定位AR码，尺寸：6cm x 6cm。

公共区

场地内除得分区以外的区域属于公共区。公共区和得分区之间以黑色线条划分，黑色线条外线以内属于得分区，外线以外属于公共区。

选手站位区

在每局比赛中，每支队伍的上场队员需站立在指定区域内，不能离开相应站位区。

得分金砖

在地图中央位置，放置了3块玩偶金砖，大小为0.1m 正方体；3块时钟金砖，大小为0.1m正方体；3块杯子金砖，大小为0.1m正方体；每块金砖相互之间间隔4.5cm。

场地制作

场地由3m*3m正方形地图构成，四周由0.6m高围栏环绕。

说明

- 1、比赛场地以承办提供的为准。
- 2、参赛机器人必须适应承办方提供的比赛场地和物料。



任务说明：

1. 比赛时间**10分钟**
2. 机器人的控制方式：自动，即使用预先编写的程序，控制机器人移动、抓取和放置金砖，程序运行后，不能触碰键盘和鼠标。
3. 比赛队伍需要在A区的收取区放置机器人
4. 比赛开始后，机器人自动导航到分类区，获取每局不同种类方块需放置位置后，自动导航到地图中间位置，抓取方块并放置到对应区域
5. 赛队需要将尽可能多的方块放置在其中B、C、D三个得分区（运输区与收取区），每局物品对应得分区可能不同，依据每局裁判随机放置的物品与区域放置分类（若第一局**玩偶**需要放置B区，第二局**玩偶**可能要放在D区）
6. 每场比赛参赛队伍有**三次**机会向裁判申请重新调试机器人，申请通过后由工作人员将机器人移回A区收取区，参赛队员方可触碰控制电脑，但计时不暂停。**当赛队抓取第一个方块后，不能向裁判申请重新调试机器人。**
7. 金砖允许堆叠。



得分细则：

金砖得分标准

当赛局结束时，如果一块金砖同时符合：①完全进入得分区；②位于地图or其它金砖的上方，同时其和地图or下方金砖的接触面与地面平行（不平行则为第一层得分）；③不超过场

地围栏的顶部（超过顶部不得分）；④机械臂完全放下得分金砖。

金砖归属

二层以上的金砖区域归属按第一层区域计算。例如：一层在收取区，二层三层都是收取区。若出现二层的金砖下方同时有收取区和运输区的金砖，则按运输区计算。

判断④机械臂完全放下得分金砖的标准

由工作人员提起比赛设备，若金砖没有随比赛设备提起，则视为得分金砖。

每局比赛三种得分金砖对应不同的B、C、D得分区，金砖放错位置不得分，且上方接触的货物或更上方间接接触的货物层数减一。

收取区得分

当一块得分金砖完全位于收取区，则其符合收取区计分规则。第N层的收取区得分金砖，每个计 $(N+2N-1)$ 分。

运输区得分

当一块得分金砖同时位于收取区和运输区或全部位于运输区，则其符合运输区计分规则。第N层的运输区得分金砖，每个计 $(2N-1)$ 分。



评分方法：

收取区得分

当一块得分金砖完全位于收取区，则其符合收取区计分规则。第N层的收取区得分金砖，每个计 $(N+2N-1)$ 分。

运输区得分

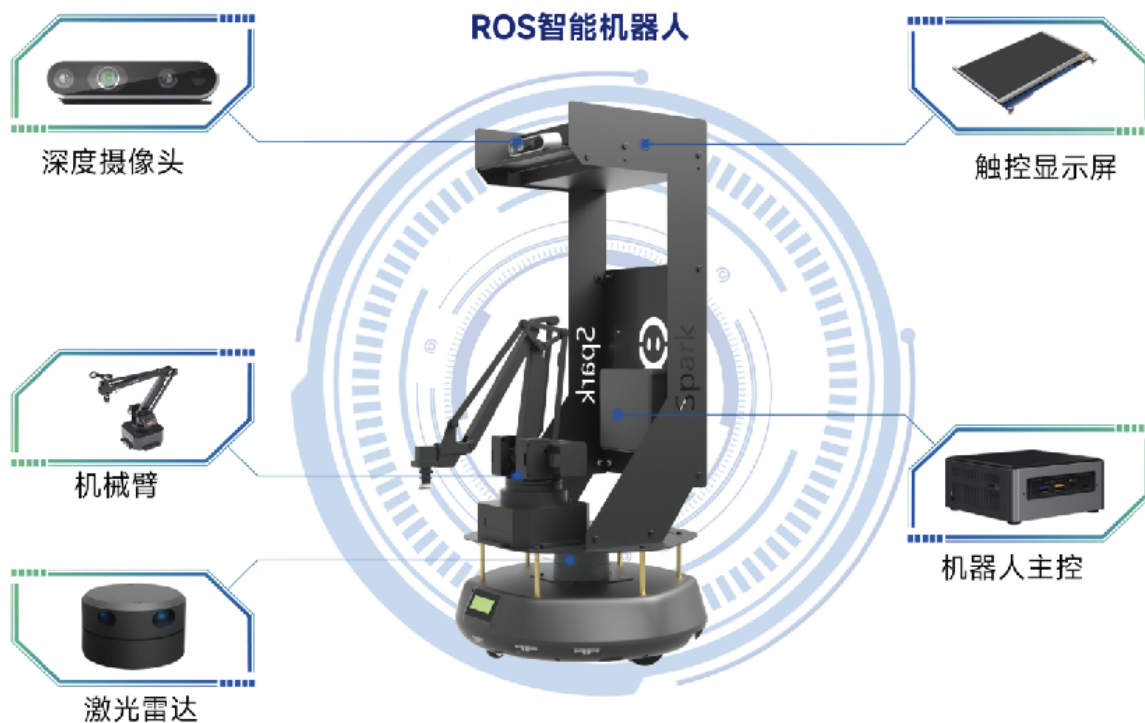
当一块得分金砖同时位于收取区和运输区或全部位于运输区，则其符合运输区计分规则。第N层的运输区得分金砖，每个计 $(2N-1)$ 分。





机器人要求：

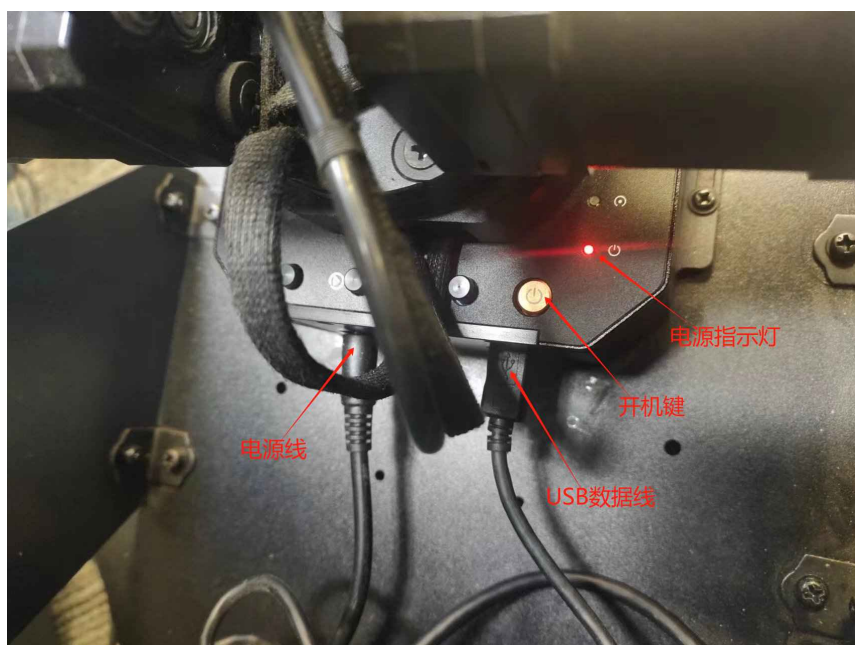
统一使用机器人**Spark-I**全套版进行比赛，硬件及结构不能进行改装，软件程序不限，但必需使用ROS。



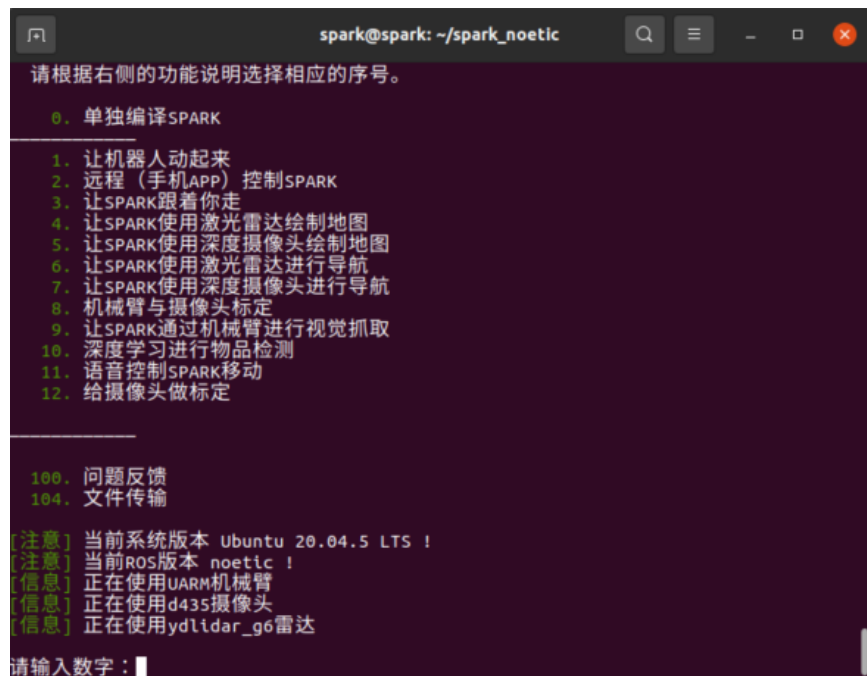
2. 比赛准备部分

2.1 手眼标定操作

标定前请确保机械臂处于开机状态，USB数据线正确连接至拓展坞，检查电源线是否松动。



- 1 打开终端输入：
- 2 `cd spark_noetic/`
- 3 `./onekey.sh`



```
spark@spark: ~/spark_noetic
请根据右侧的功能说明选择相应的序号。

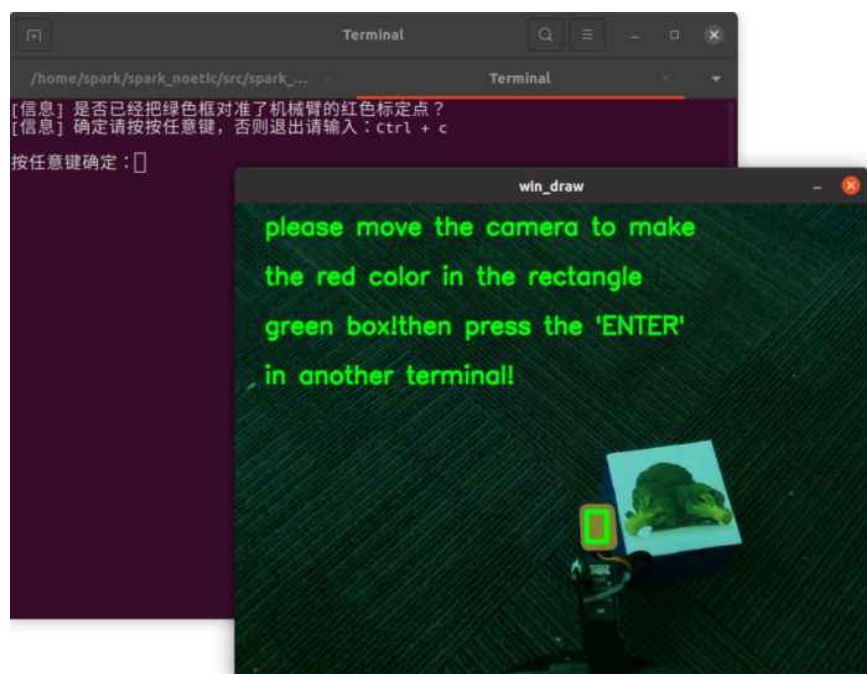
0. 单独编译SPARK
1. 让机器人动起来
2. 远程（手机APP）控制SPARK
3. 让SPARK跟着你走
4. 让SPARK使用激光雷达绘制地图
5. 让SPARK使用深度摄像头绘制地图
6. 让SPARK使用激光雷达进行导航
7. 让SPARK使用深度摄像头进行导航
8. 机械臂与摄像头标定
9. 让SPARK通过机械臂进行视觉抓取
10. 深度学习进行物品检测
11. 语音控制SPARK移动
12. 给摄像头做标定

100. 问题反馈
104. 文件传输

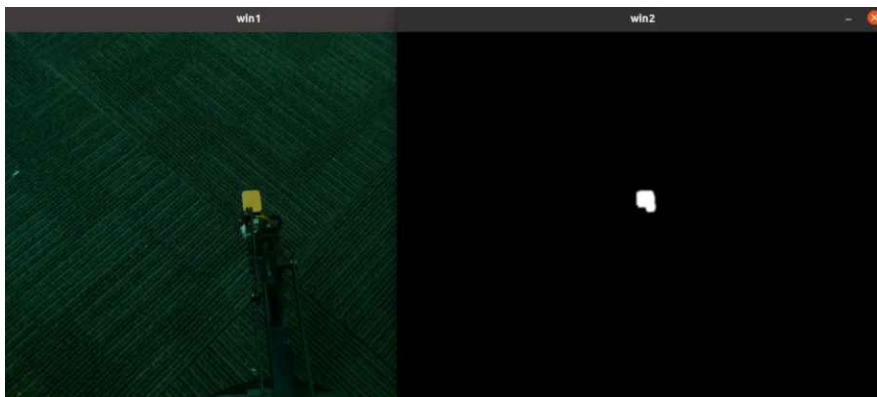
[注意] 当前系统版本 Ubuntu 20.04.5 LTS !
[注意] 当前ROS版本 noetic !
[信息] 正在使用UARM机械臂
[信息] 正在使用d435摄像头
[信息] 正在使用ydlidar_g6雷达

请输入数字： 
```

- 1 输入数字：8
- 2 选择当前Spark机器人使用的机械臂型号：uArm
- 3 点击回车后进入校准界面



- 1 根据提示旋转相机，使其绿色图案对准机械臂上的红色方块
- 2 对准后在终端界面回车确定



- 1 标定过程中请勿遮挡机械臂上的红色方块，确保右侧二值化图像时刻显示。

```

/home/spark/spark_noetic/src/spark_app/spark_carry/launch...
start to move the uarm---
[ WARN] [1704683850.412056312]: Joint state with name: "Joint1" was received but
not found in URDF
180 200
1/20,pose x,y: 180,200. cam x,y: 101 322
[ WARN] [1704683860.461714189]: Joint state with name: "Joint1" was received but
not found in URDF
185 190
2/20,pose x,y: 185,190. cam x,y: 114 315
190 180
3/20,pose x,y: 190,180. cam x,y: 128 308
[ WARN] [1704683870.512748968]: Joint state with name: "Joint1" was received but
not found in URDF
195 170
4/20,pose x,y: 195,170. cam x,y: 140 302
200 160
5/20,pose x,y: 200,160. cam x,y: 154 296
[ WARN] [1704683880.561781551]: Joint state with name: "Joint1" was received but
not found in URDF
205 150
6/20,pose x,y: 205,150. cam x,y: 167 290
210 140
7/20,pose x,y: 210,140. cam x,y: 181 284
[ WARN] [1704683890.561862585]: Joint state with name: "Joint1" was received but
not found in URDF
215 130
8/20,pose x,y: 215,130. cam x,y: 193 277
220 120
9/20,pose x,y: 220,120. cam x,y: 207 271
  
```

- 💡 标定结束后，机械臂自动归位
- 终端输入 `ctrl + c` 查看标定结果
- 需要注意红色选框内的值，每个值之间相差不能过大，例如第一个值与第二个值之间相差10，但第二个值与第三个值之间相差30，则说明标定过程异常，需要重新开始标定。
- 标定文件默认存放路径：`~/thefile.txt`。

2.2 标定过程异常解析

受当前场地环境光影响，使得标定程序无法识别红色方块，右侧二值化图像呈闪烁或消失不见的状态。

解决方法：可尝试将Spark机器人放至环境光较弱或背景较为纯色的地方，再次进行标定操作。

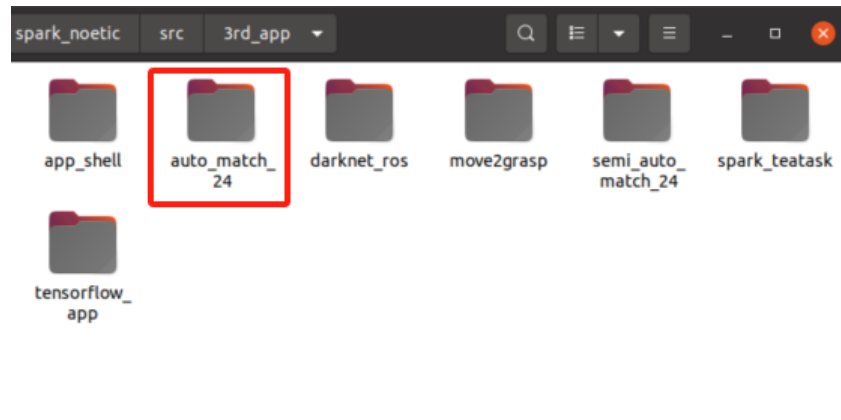
2.3 自动赛必要文件

功能包：auto_match

节点文件：auto_match.launch

2.4 安装功能包

- 1 建议将功能包安装至此目录下：
- 2 `sudo unzip auto_match_24.zip -d /home/spark/spark_noetic/src/3rd_app/`
- 3 输入密码：spark
- 4 `cd spark_noetic`
- 5 `catkin_make`



2.5 添加权限

- 1 `cd ~/spark_noetic/src/3rd_app/auto_match_24/auto_match/nodes`
- 2 `sudo chmod +x *.py`
- 3 `cd ../scripts`
- 4 `sudo chmod +x *.sh`

```
spark@spark:~/spark_noetic/src/3rd_app/auto_match_24/auto_match/nodes$ ll
total 60
drwxrwxr-x 2 spark spark 4096 1月 12 17:32 ./
drwxrwxr-x 7 spark spark 4096 1月 12 17:32 ../
-rwxrwxr-x 1 spark spark 8505 1月 3 07:04 call_node.py*
-rwxrwxr-x 1 spark spark 6581 1月 3 07:25 detect_ros.py*
-rwxrwxr-x 1 spark spark 4306 1月 5 08:37 mark_move.py*
-rwxrwxr-x 1 spark spark 21439 1月 12 09:20 runtask.py*
```

```
spark@spark:~/spark_noetic/src/3rd_app/auto_match_24/auto_match/scripts$ ll
total 28
drwxrwxr-x 4 spark spark 4096 1月 12 17:32 ./
drwxrwxr-x 7 spark spark 4096 1月 12 17:32 ../
-rwxrwxr-x 1 spark spark 110 1月 5 07:01 cmd_location_learn_guide.sh*
drwxrwxr-x 3 spark spark 4096 1月 12 17:32 .idea/
-rwxrwxr-x 1 spark spark 7273 1月 11 05:58 location_learn_guide.sh*
drwxrwxr-x 2 spark spark 4096 1月 12 17:32 .vscode/
```

2.6 地图构建

- 1 `cd spark_noetic/`
- 2 `source devel/setup.bash`

```
3 roslaunch auto_match auto_match.launch open_guide:=true
```

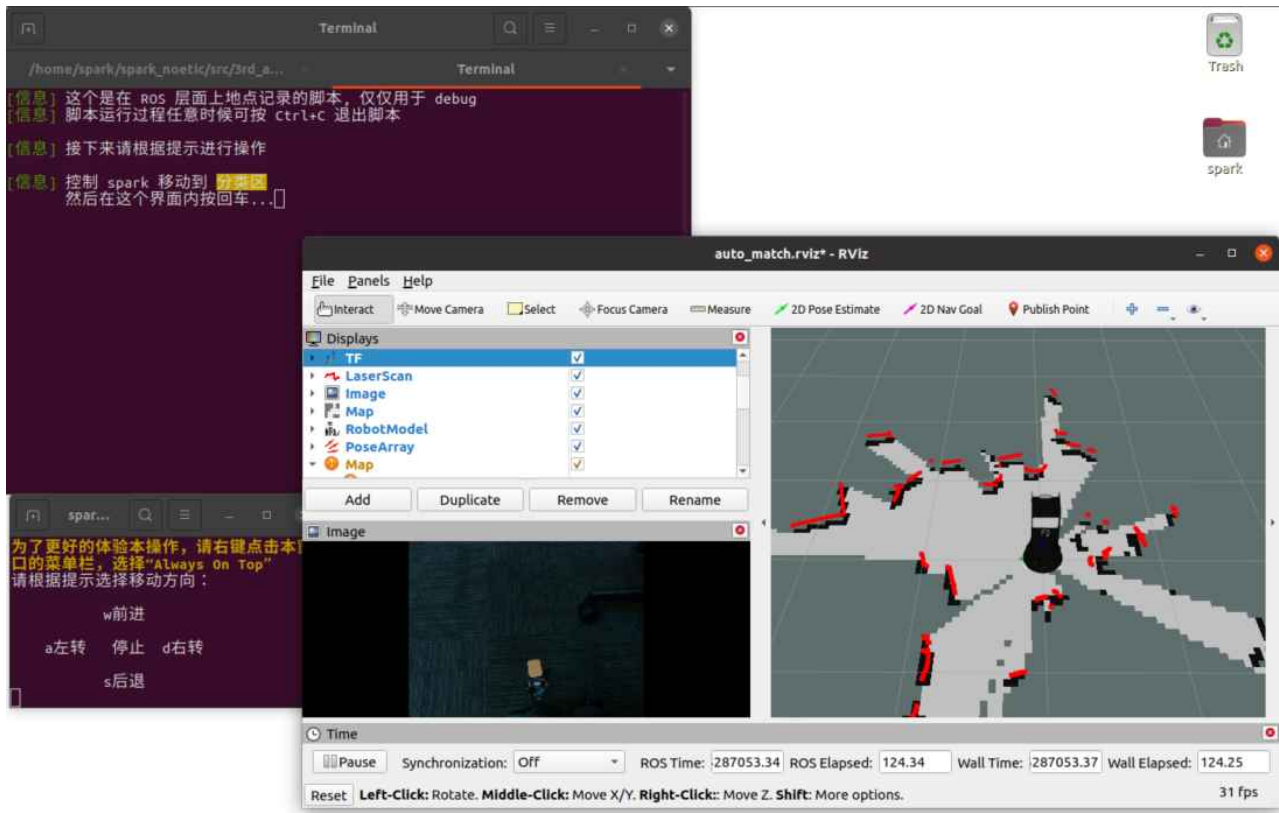


根据终端提示：

WASD：机器人底盘移动。

Enter：记录机器人所在区域点位\保存地图。

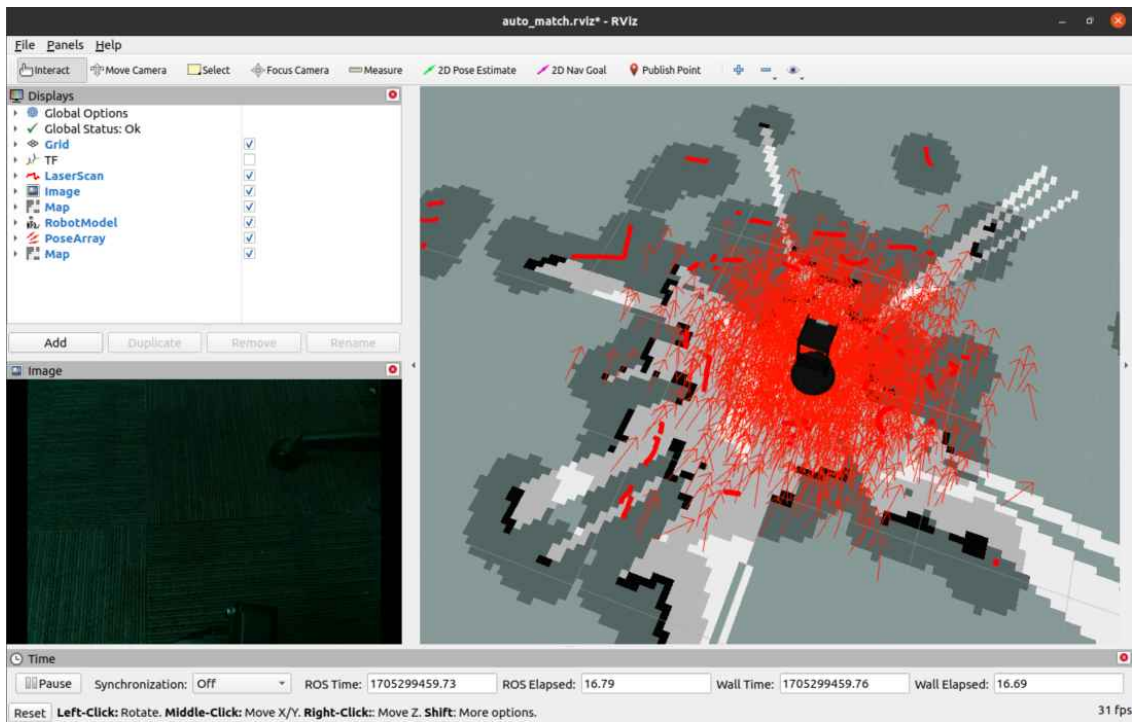
注意：需要记录完所有点位才可保存栅格地图。



运行效果

2.7 任务开始

- 1 `cd spark_noetic/`
- 2 `source devel/setup.bash`
- 3 `roslaunch auto_match auto_match.launch`
- 4 另起一个终端：
- 5 `rostopic pub /task_start_flag std_msgs/String "data: 'true'" -1 >/dev/null`



运行效果

3. 自动赛程序分析

在地图构建阶段记录分类区、收取区和分拣区的导航点位置。任务开始后，机器人首先前往分类区识别需要抓取的物体，随后前往分拣区对物体进行抓取，并自动前往被抓取物体对应的收取区。

3.1 执行文件

物体识别：detect_ros.py

底盘导航：mark_move.py

抓取逻辑：runtask.py

3.2 构建比赛地图，获取导航点位置

使用Gmapping算法对场地进行二维建图，建图过程中手动标定任务目标点。

```
1 # auto_match.launch: 60行
2 <!-- =====一开始准备记录地点===== -->
3 <group if="$(arg open_guide)">
4
5     <!-- SLAM: Gmapping, Cartographer, Hector, Karto, Frontier_exploration-->
6     <include file="$(find spark_slam)/launch/spark_$(arg slam_methods).launch">
7         <arg name="configuration_basename" value="$(arg
            configuration_basename)"/>
```

```

8     </include>
9
10    <!-- 地点记录引导脚本 -->
11    <node pkg="auto_match" type="cmd_location_learn_guide.sh"
      name="cmd_location_learn_guide" />
12
13    <!--创建新的终端，spark键盘控制 “wsad”分别代表“前后左右”-->
14    <node pkg="spark_teleop" type="keyboard_control.sh" name="kc" if="$(arg
      open_guide)"/>
15
16 </group>

```

地点记录脚本：

```

1  #!/bin/bash
2
3  Green_font_prefix="\033[32m" && Red_font_prefix="\033[31m" &&
  Green_background_prefix="\033[42;37m" && Red_background_prefix="\033[41;37m"
  && Yellow_background_prefix="\033[43;37m" && Font_color_suffix="\033[0m" &&
  Yellow_font_prefix="\e[1;33m" && Blue_font_prefix="\e[0;34m"
4  Info="${Green_font_prefix}[信息]${Font_color_suffix}"
5  echo -e "${Info} 这个是在 ROS 层面上地点记录的脚本，仅仅用于 debug"
6  echo -e "${Info} 脚本运行过程任意时候可按 Ctrl+C 退出脚本"
7  echo -e ""
8  echo -e "${Info} 接下来请根据提示进行操作"
9
10 echo -e ""
11 echo -e "${Info} 控制 spark 移动到 ${Yellow_background_prefix}分类区
    ${Font_color_suffix}\c"
12 echo && stty erase '^H' && read -p "          然后在这个界面内按回车..."
13 rostopic pub /mark_nav std_msgs/String "data: 'learn Classification_area'" -1
    >/dev/null
14 echo -e "${Info} 地点名为
    ${Yellow_background_prefix}Classification_area${Font_color_suffix}"
15
16 echo -e ""
17 echo -e "${Info} 控制 spark 移动到 ${Yellow_background_prefix}收取区
    B${Font_color_suffix}\c"
18 echo && stty erase '^H' && read -p "          然后在这个界面内按回车..."
19 rostopic pub /mark_nav std_msgs/String "data: 'learn Collection_B'" -1
    >/dev/null
20 echo -e "${Info} 地点名为
    ${Yellow_background_prefix}Collection_B${Font_color_suffix}"
21

```

```

22
23 echo -e ""
24 echo -e "${Info} 控制 spark 移动到 ${Yellow_background_prefix}分拣区北
    ${Font_color_suffix}\c"
25 echo && stty erase '^H' && read -p "          然后在这个界面内按回车..."
26 rostopic pub /mark_nav std_msgs/String "data: 'learn Sorting_N'" -1 >/dev/null
27 echo -e "${Info} 地点名为
    ${Yellow_background_prefix}Sorting_N${Font_color_suffix}"
28
29 echo -e ""
30 echo -e "${Info} 控制 spark 移动到 ${Yellow_background_prefix}分拣区西
    ${Font_color_suffix}\c"
31 echo && stty erase '^H' && read -p "          然后在这个界面内按回车..."
32 rostopic pub /mark_nav std_msgs/String "data: 'learn Sorting_W'" -1 >/dev/null
33 echo -e "${Info} 地点名为
    ${Yellow_background_prefix}Sorting_W${Font_color_suffix}"
34
35
36 echo -e ""
37 echo -e "${Info} 是否标记完成,保存地图? \c"
38 echo && stty erase '^H' && read -p "          然后在这个界面内按回车..."
39 rostopic pub /mark_nav std_msgs/String "data: 'finish'" -1 >/dev/null
40 # 保存地图
41 CURRENTPATH=$(cd `dirname $0`; pwd)
42 gnome-terminal -x bash -c "roslaunch map_server map_saver -f
    $CURRENTPATH/./config/test_map"
43 echo -e "${Info} 结束标记地点,已保存地图"
44
45
46 echo -e ""
47 echo -e "${Info} 地点记录已完成\c"
48 echo -e ""
49 echo && stty erase '^H' && read -p "现在可以按回车退出脚本..."

```

```

1 rostopic pub /mark_nav std_msgs/String "data: 'learn Classification_area'" -1
  >/dev/null

```



解析：发布一个话题名为 `/mark_nav` 的自定义消息话题，内容 `“learn Classification_area”`，作为记录分类区点位的信号。

```

1 def mark_nav(self,mark_name):
2     '''
3     设定标定地点，与后面导航相关，采用字典的方式存放
4     建议标记地点名字,分类区、分拣区、收取区
5     '''
6
7     rospack = rospkg.RosPack()
8     mark_map_path = os.path.join(rospack.get_path('auto_match'),'config',
9     'mark_map.pkl') # 标记地图yaml文件地址
10
11     # 转化成字符型变量
12     mark_name = str(mark_name)
13     # 结束地点标记
14     if str.find(mark_name,str("finish"))!= -1:
15         # 将字典信息写入 pkl 文件
16         with open(mark_map_path, 'wb') as file:
17             pickle.dump(dict_mark, file)
18         print("已保存pkl文件,文件保存在:",mark_map_path)
19         print("当前字典内容: \n",dict_mark)
20
21     # 前往地点
22     if str.find(mark_name,str("go"))!= -1:
23         # 从文件中加载数据
24         with open(mark_map_path, 'rb') as file:
25             self.mark_map = pickle.load(file)
26         # 输出加载的数据
27         print("mark_dict=",self.mark_map)
28         # 提取mark名称
29         mark_name = (mark_name.split())[2]
30         # 判断地点位置是否在地点字典中
31         if mark_name in self.mark_map:
32             self.navigation(mark_name)
33         else:
34             print("此地点尚未登陆在字典中")
35
36     # 学习地点
37     if str.find(mark_name,str("learn"))!= -1:
38         # 提取mark名称
39         mark_name = (mark_name.split())[2]
40         # 获取当前位置
41         self.current_position = self.get_currect_pose()
42         # 标记当前位置
43         dict_mark[mark_name] = self.current_position
44         print("当前字典内容: \n",dict_mark)

```

💡 `find(mark_name, str("learn"))`：查找 `mark_name` 中是否包含“learn”字符串，不包含则返回 `-1`。

`pickle.dump()`：`pickle` 模块提供的函数之一，用于将对象序列化并保存到文件中。

`pickle.load()`：用于从文件中加载已经序列化的对象。

💡 解析：`mark_name` 话题用于接收来自记录点位、完成记录和导航目标点三个信息。

1. 当接收到“learn”开头的消息，意味着要记录机器人当前点位，便以键值对的方式将目标点名称和点位信息写入字典。
2. “finsh”：则代表点位记录已完成，将所有数据保存到pkl文件。
3. “go”：接收到导航目标点(键)，查找pkl文件里与当前键对应的值，即点位信息，并发送。

获取机器人点位：

```
1 def get_currect_pose(self):
2     (target_trans, target_rot) = self.listener.lookupTransform("map",
3     "base_footprint", rospy.Time(0))
4     print("target_trans:", target_trans)
5     print("target_rot:", target_rot)
6     return tf_transformations.compose_matrix(translate=target_trans,
7     angles=tf_transformations.euler_from_quaternion(target_rot))
```

💡 解析：先获取机器人在 `"map"` 坐标系到 `"base_footprint"` 坐标系的变换信息（位姿）。再通过 `compose_matrix` 函数，将获取到的平移和旋转信息组合成一个变换矩阵，并将其作为结果返回。

导航目标点发送：

```
1 def navigation(self, mark_name):
2     '''
3     根据地点进行导航
4     '''
5     print("start navigation")
```

```

6      # movebase初始化
7      goal = MoveBaseGoal()
8      goal.target_pose.header.frame_id = 'map'
9      goal.target_pose.header.stamp = rospy.Time.now()
10     # 设定目标地点
11     self.drop_position = self.mark_map[mark_name]
12     tran = tf_transformations.translation_from_matrix(self.drop_position)
13     quat = tf_transformations.quaternion_from_matrix(self.drop_position)
14     pose = Pose(Point(tran[0], tran[1], tran[2]), Quaternion(quat[0], quat[1],
    quat[2], quat[3]))
15     goal.target_pose.pose = pose
16     # 把目标位置发送给MoveBaseAction的服务器
17     self.move_base.send_goal(goal)

```

-  1. `MoveBaseGoal()`：包含导航任务设置目标点的坐标、姿态、路径规划方式等信息。
2. `mark_name`：一般接收的是目标点名称，如：`Classification_area`。
3. `translation_from_matrix()`：用于从变换矩阵中提取平移向量。变换矩阵通常包含了平移、旋转和缩放等变换信息。这个函数会返回一个包含三个元素的向量，表示在三个坐标轴上的平移。
4. `quaternion_from_matrix()`：用于从变换矩阵中提取四元数。四元数通常用于表示旋转的信息。这个函数会返回一个包含四个元素的列表，表示四元数的 `[x, y, z, w]` 分量。
5. `send_goal()`：发送导航目标点。

3.3 导航到分类区，获取放置物体地址

按照识别到物体的位置信息，根据由左到右，确定放置区域。

```

1      # ===== 现在开始执行任务 =====
2      rospy.loginfo("start task now.")
3
4      # ===== 离开起始区,避免在膨胀区域中, 导致导航失败 =====
5      self.robot.step_go(0.2)
6      if self.stop_flag: return
7
8      # ===== 移动机械臂 =====
9      self.arm.arm_position_reset() # 重置机械臂坐标系
10     self.arm.arm_grasp_ready() # 移动机械臂到其他地方
11

```



```

12     if self.robot.goto_local("Classification_area"):
13         rospy.sleep(2)
14     else :
15         rospy.logerr("Navigation to Classification_area failed,please run task
again ")
16         self.stop_flag = True

```

 **解析：**程序开始时，机器人为避免陷入膨胀区域，会向前走20厘米。然而这时机器人定位坐标是错乱的，但导航目标点已发出，DWA 收到无法到达的目标点则会以自身旋转的方式等待前进的时机。因为 DWA 这一特性，使得 AMCL 在机器人自转过程中得以修正机器人在场地上的定位坐标。

 **解析2：**当机器人导航至分类区时会返回一个 True ，执行物体识别。
导航失败，则程序结束。

目标点导航

```

1 def goto_local(self, name):
2     '''
3     根据目标点名称,发布目标位置到MoveBase服务器,根据返回状态进行判断
4     @return: True 为成功到达, False 为失败
5     '''
6
7     # 发布目标位置
8     self.goto_local_pub.publish("go "+name)
9
10    # 设定1分钟的时间限制, 进行阻塞等待
11    try:
12        ret_status = rospy.wait_for_message(
13            'move_base/result', MoveBaseActionResult,
14            rospy.Duration(60)).status.status
15    except Exception:
16        rospy.logwarn("nav timeout!!!")
17        ret_status = GoalStatus.ABORTED
18
19    # 如果一分钟之内没有到达, 放弃目标
20    if ret_status != GoalStatus.SUCCEEDED:
21        rospy.Publisher("move_base/cancel", GoalID, queue_size=1).publish(
22            GoalID(stamp=rospy.Time.from_sec(0.0), id=""))
23    try:

```

```

23         rospy.wait_for_message(
24             'move_base/result', MoveBaseActionResult, rospy.Duration(3))
25     except Exception:
26         rospy.logwarn("move_base result timeout. this is abnormal.")
27         rospy.loginfo("====Timed out achieving goal====")
28         return False
29     else:
30         rospy.loginfo("====Goal succeeded====")
31         return True

```

- 💡 1. `name`：机器人要到达的目标地点。
2. `wait_for_message`：等待ROS话题上的消息，直到接收指定获取的ROS话题或超出所设定的等待时长。
3. `move_base/result`：`MoveBaseActionResult` 消息话题，用于获取机器人在导航任务执行的结果。
4. `Duration()`：设置等待时长。
5. `GoalStatus.ABORTED`：导航任务的执行状态，表示任务被取消。
6. `GoalStatus.SUCCEEDED`：表示任务已成功。
7. `move_base/cancel`：`GoalID`消息话题，用于发送取消导航任务的请求。

💡 **解析：**如果机器人的导航任务执行超过设定的60秒，则直接取消本次导航任务。

3.4 导航到分拣区，识别并抓取物体

机器人以北西的顺序前往分拣区，每经过一个区自动抓取一块物体。

```

1     # runtask.py: 496行
2     # 创建任务安排字典，设定前往的抓取地点与次数
3     sorting_status_times = {
4         "Sorting_N":1,
5         "Sorting_W":1,
6     }
7     sorting_name = "Sorting_N"
8
9     # =====开始循环运行前往中心区域抓取与放置任务=====
10    while True:
11        # 根据任务安排逐步执行

```

```

12     print("readying to sort_areas")
13     if sorting_status_times[sorting_name] == 0:
14         sorting_status_times.pop(sorting_name)
15         if len(sorting_status_times) == 0:
16             break
17         else:
18             sorting_name = list(sorting_status_times.keys())[0]
19
20     # =====导航到目标地点=====
21
22     ret = self.robot.goto_local(sorting_name) # 导航到目标点
23     rospy.sleep(2) # 停稳
24
25     if self.stop_flag: return
26
27     # =====识别并抓取物体=====
28     item_type = 0
29     if ret: # 判断是否成功到达目标点
30         print("=====扫描中, 准备抓取===== ")
31         item_type = self.arm.grasp() # 抓取物品并返回抓取物品的类型
32         print("=====向后退一点===== ")
33         self.robot.step_back() # 后退
34         if self.stop_flag: return
35
36     if item_type == 0: # 如果没有识别到物体, 将该地点的抓取次数归0
37         sorting_status_times[sorting_name] = 0
38         continue

```



1. `goto_local()` : 导航到指定目标点, 并返回导航任务执行结果。
2. `grasp()` : 用于执行物体识别抓取任务, 并返回被抓取物体信息的列表。



解析: 分类区任务结束, 机器人首先会导航到分拣区北方向记录的坐标, 待机器人停稳, 机械臂自动抓取识别率最高的物体, 随后根据抓取的物体自动前往与物体对应的收取区。

3.5 导航到收取区, 放置并堆叠物体

根据抓取的物体类别自动前往相应的收取区。

```

1     # runtask.py: 539行
2     # =====放置物品=====

```

```

3     self.arm.arm_grasp_ready()
4     print("=====>前往放置区==== ")
5     ret = self.robot.goto_local("Collection_B") # 根据抓到的物品类型，导航到对应的放置区
6     rospy.sleep(2) # 停稳
7     if self.stop_flag: return
8
9     if ret:
10         self.arm.drop() # 放下物品
11         self.robot.step_back() # 后退
12         if self.stop_flag: return
13     else:
14         rospy.logerr("task error: navigation to the drop_place fails")
15         self.arm.drop()
16         if self.stop_flag: return

```



1. `arm_grasp_ready()`：将机械臂移动到指定位置。
2. `item_type`：被抓取物体的ID号。



解析：机器人取到物体后根据当前物体信息，自动前往对应收取地点卸货。

3.6 抓取物体部分

使用深度学习找到所需物品在图像上的位置, 估算物品实际位置, 让机械臂抓取。

```

1 # runtask.py: 201行
2 def grasp(self):
3     '''
4     使用深度学习找到所需物品在图像上的位置，估算物品实际位置，让机械臂抓取
5     @return: 抓取到物品的id, 0为未识别到需要的物品
6     '''
7     r1 = rospy.Rate(0.25)
8     r2 = rospy.Rate(10)
9     x = 0
10    y = 0
11    z = 0
12
13    # 寻找物品
14    cube_list = self.cam.detector()
15    # rospy.sleep(2)

```

```

16     if len(cube_list) == 0:
17         rospy.logwarn("没有找到物品啊。。。去下一个地方")
18         self.grasp_status_pub.publish(String("1"))
19         return 0
20
21     # 获取机械臂目标位置
22     x = self.x_kb[0] * cube_list[0][1][1] + self.x_kb[1]
23     y = self.y_kb[0] * cube_list[0][1][0] + self.y_kb[1]
24     z = -55
25
26     print(f"找到物品了! 它在: {x}, {y}, {z}")
27
28     # 机械臂移动到目标位置上方
29     self.interface.set_pose(x, y, z + 10)
30     rospy.sleep(2)
31
32     # 机械臂移动到目标位置
33     self.interface.set_pose(x, y, z)
34     rospy.sleep(2)
35
36     # 打开气泵, 进行吸取
37     self.interface.set_pump(True)
38     rospy.sleep(1)
39     # r2.sleep()
40
41     # 抬起目标方块
42     print(f"我把物品抬起来了")
43     self.interface.set_pose(x, y, z + 80)
44     rospy.sleep(2)
45     # r1.sleep()
46
47     self.grasp_status_pub.publish(String("0"))
48
49     return cube_list[0][0]

```

- 💡 1. `Rate()` : ROS中用于控制循环频率的函数, 参数为 `10` 表示设置循环的频率为10Hz。
- 2. `set_pose()` : 用于控制机械臂运动, `xyz` 三个参数代表机械臂末端坐标。
- 3. `set_pump()` : 控制气泵开和关。

```

1     x = self.x_kb[0] * cube_list[0][1][1] + self.x_kb[1]
2     y = self.y_kb[0] * cube_list[0][1][0] + self.y_kb[1]

```

- 
1. `x_kb` 和 `y_kb` 是通过手眼标定得到的标定数据。
 2. `cube_list[0][1][0]`、`cube_list[0][1][1]` 分别为物体中心点 `xy` 坐标。
 3. 此步骤目的是将摄像头图像上的像素坐标转换为实际物体在机械臂坐标系中的坐标。